

```

elif action == 'w':
    print('Withdraw:')

    userWithdrawAmount = input('Please enter the amount to withdraw: ')
    userWithdrawAmount = int(userWithdrawAmount)
    userPassword = input('Please enter the password: ')

    if userWithdrawAmount < 0:
        print('You cannot withdraw a negative amount')

    elif userPassword != accountPassword:
        print('Incorrect password for this account')

    elif userWithdrawAmount > accountBalance:
        print('You cannot withdraw more than you have in your account')

    else: #OK
        accountBalance = accountBalance - userWithdrawAmount
        print('Your new balance is:', accountBalance)

print('Done')

```

Listing 1-2: Bank simulation for a single account

The program starts off by initializing three variables to represent the data of one account ❶. Then it displays a menu that allows a choice of operations ❷. The main code of the program acts directly on the global account variables.

In this example, all the actions are at the main level; there are no functions in the code. The program works fine, but it may seem a little long. A typical approach to make longer programs clearer is to move related code into functions and make calls to those functions. We'll explore that in the next implementation of the banking program.

Implementation 2—Single Account with Functions

In the version of the program in [Listing 1-3](#), the code is broken up into separate functions, one for each action. Again, this simulation is for a single account.